

Performance & Cost Optimization



Learn2
CloudData

Powered by

www.learn2clouddata.com

Sujith Nair

Cloud Data Architect
Snowflake Snowpro Certified

Section Introduction–Performance & Cost Optimization

- What is Data caching ?
- LAB 1–Use the Snowflake Data Cache
- What is Table Partitioning
- What are Micro-partitions in Snowflake.
- Clustering Tables in Snowflake
- LAB 2–Cluster a Snowflake table
- Clustered Materialized views
- LAB 3–Create Clustered Materialized view.
- Scaling up vs Scaling out
- LAB 4–Scaling up and Scaling out in Action.
- What is a scaling policy ?
- LAB 5–Create and modify scaling policy.
- What is Query Acceleration Service ?
- What is Scale Factor ?
- LAB 6–Query Acceleration.
- Summary.

What is Data caching ?

Data caching is the process of temporarily storing query results or frequently accessed data in memory for reuse by queries leading to time and credit saving and more efficient use of resources.

Types of cache in Snowflake.

- Result cache.
- Local disk Cache.
- Metadata cache.



What is Data caching ?

Results Cache.

- The results cache stores the results of queries that have been previously executed.
- An identical query submitted again will use the cache if the underlying data hasn't changed. Snowflake returns results directly from the result cache bypassing the need to re-execute the query.
- Fetching data from the results cache does not consume credits and is much faster.
- Cached results are stored for 24 hours by default, but the cache is invalidated if the underlying data changes.
- Data from results cache can be re-used across different session in the same account.
- If the order of the columns or case in the query changes the result cache will not be used because the query is not identical.
- Queries which have functions that produce nondeterministic results like `CURRENT_TIMESTAMP` cannot use the result cache.
- Queries in the result cache are sometimes referred to as persisted query results.
- The results cache is in the Cloud services layer of Snowflake.
- The benefit of the results cache being in the cloud services layer of snowflake is that it is independent of any component that can be turned off leading to the cache being lost.

What is Data caching ?

Local disk Cache.

- The Local disk cache stores data that has been read from Snowflake cloud storage during query execution.(Eg: COPY)
- The Local disk cache also stores data that has been read from Snowflake tables as part of a query because the tables data is stored in s3.
- Local disk cache is in the SSD(Hard Drive) storage of the virtual warehouse.
- The main difference between Local disk cache and result cache is that Local disk cache is designed to cache data that may need to be re-read from AWS s3 whereas the Query results cache is designed to cache query results.
- Now Query results can be different from the data read from s3 due to SQL functions and computation applied to the data being read.
- Snowflake looks for requested data in the Local disk cache if and only if it is not able to find it in the results cache.
- If data is not found in either of the two caches then it will go back to s3 to fetch the data.
- Intermediate query results which is the temporary data produced at various stages during the execution of a complex query is stored in the Local Disk Cache for use if needed.
- The local disk cache persists as long as the virtual warehouse is running. The local disk cache is lost when the warehouse is suspended or deleted.

What is Data caching ?

Metadata Cache.

- The metadata cache stores information like table names and their column names in the cache.
- Table statistics like number of rows in the table, MIN and MAX values for each column are stored. Row count like `SELECT COUNT(*) from <table>` are made available from the metadata cache instead of the table.
- Object dependencies are stored in the metadata cache.
- Metadata cache reduces time needed to parse the query as references to tables and columns are available in the cache.
- Metadata cache leads to performance improvement due to the need to not access the database storage layer to get information for parsing or row counts.

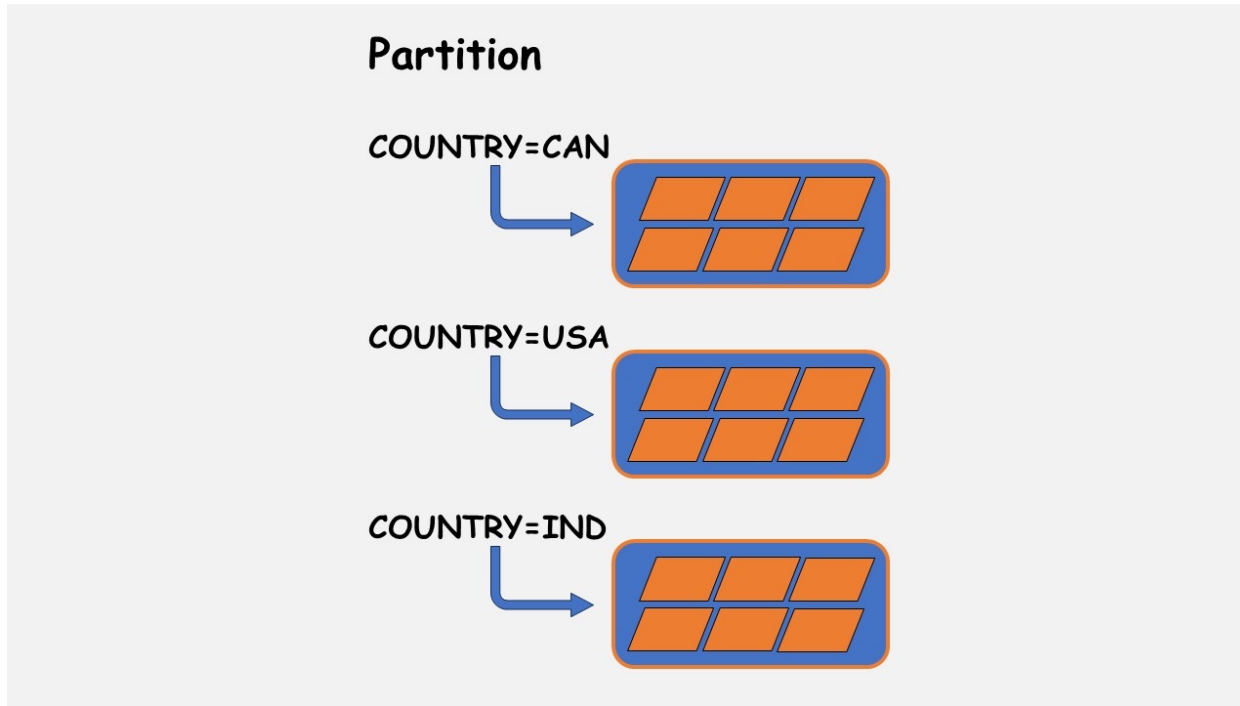
What is a Table partitioning ?

Table partitioning is a database optimization technique that involves dividing a large table into smaller, more manageable pieces called partitions. Each partition holds a subset of the table's data, based on specific criteria .

Partitioning improves query performance, enhance manageability and optimize storage by allowing the database system to perform operations on only the relevant partitions rather than scanning the entire table.



What is a Table partitioning ?



- Table partitioning leads to partition pruning.
- Improved performance.

What are Micro-partitions in Snowflake ?

A Micro-partition in Snowflake is a unit of data storage that efficiently organizes, manages and stores data. Snowflake automatically divides table data into micro-partitions when data is being loaded.

These micro-partitions are stored in columnar storage, allowing for efficient compression and faster query performance.

Key Characteristics of Micro-Partitions:

Immutable and Versioned: Micro-partitions are immutable, meaning once they are created they do not change and they cannot be modified. New data or updates result in new Micro-partitions being created. This immutability supports features like Time Travel where Snowflake can track historical versions of data.

Automatic Partitioning: Snowflake automatically creates Micro-partitions based on its analysis of the data. There is no need for manual partitioning, as Snowflake manages this process.

To determine how to partition the data, snowflake analyzes the data as it is being loaded to determine which column would be most suitable for partitioning.

This partitioning is primarily based on how the data is naturally clustered during ingestion, such as the order in which the data is loaded. For example, if data related to a specific country are loaded together these records are likely to be stored in the same micro-partitions.

What are micro-partitions in Snowflake ?

Columnar Storage: Data within each micro-partition is stored in a columnar format, which means that each column's data is stored together. This allows for efficient compression and faster retrieval, especially for queries that only need specific columns.

This compression reduces storage costs and enhances performance by minimizing the amount of data that needs to be read during queries.

Metadata and Pruning: Snowflake stores metadata about each micro-partition including information on the range of values for each column within the partition such as minimum and maximum values for each column, row counts and more. This metadata is crucial for query optimization. This metadata allows Snowflake to perform partition pruning, which means that only relevant micro-partitions are scanned during query execution improving performance.

Size: Each micro-partition typically ranges from 50 MB to 500 MB of uncompressed data. The size can vary depending on the data's nature and the compression applied.

What are micro-partitions in Snowflake ?

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NBR	HIRE_DATE
100	STEVEN	KING	SKING	515.123.4567	2003-06-17
101	NEENA	KOCHHAR	NKOCHHAR	515.123.4568	2005-09-21
102	LEX	DE HAAN	LDEHAAN	515.123.4569	2001-01-13
105	DAVID	AUSTIN	DAUSTIN	590.423.4569	2005-06-25
106	VALLI	PATABALLA	VPATABAL	590.423.4560	2006-02-05
107	DIANA	LORENTZ	DLORENTZ	590.423.5567	2007-02-07
108	NANCY	GREENBERG	NGREENBE	515.124.4569	2002-08-17
109	DANIEL	FAVIET	DFAVIET	515.124.4169	2002-08-16
110	JOHN	CHEN	JCHEN	515.124.4269	2005-09-28
111	ISMAEL	SCIARRA	ISCIARRA	515.124.4369	2005-09-30
112	JOSE MANUEL	URMAN	JMURMAN	515.124.4469	2006-03-07
113	LUIS	POPP	LPOPP	515.124.4567	2007-12-07
114	DEN	RAPHAELY	DRAPHEAL	515.127.4561	2002-12-07
115	ALEXANDER	KHOO	AKHOO	515.127.4562	2003-05-18
116	SHELLI	BAIDA	SBAIDA	515.127.4563	2005-12-24
117	SIGAL	TOBIAS	STOBIAS	515.127.4564	2005-07-24
118	GUY	HIMURO	GHIMURO	515.127.4565	2006-11-15
119	KAREN	COLMENARES	KCOLMENA	515.127.4566	2007-08-10
120	MATTHEW	WEISS	MWEISS	650.123.1234	2004-07-18
121	ADAM	FRIPP	AFRIPP	650.123.2234	2005-04-10

What are micro-partitions in Snowflake ?

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NBR	HIRE_DATE
100	STEVEN	KING	SKING	515.123.4567	2003-06-17
101	NEENA	KOCHHAR	NKOCHHAR	515.123.4568	2005-09-21
102	LEX	DE HAAN	LDEHAAN	515.123.4569	2001-01-13
105	DAVID	AUSTIN	DAUSTIN	590.423.4569	2005-06-25
106	VALLI	PATABALLA	VPATABAL	590.423.4560	2006-02-05
107	DIANA	LORENTZ	DLORENTZ	590.423.5567	2007-02-07
108	NANCY	GREENBERG	NGREENBE	515.124.4569	2002-08-17
109	DANIEL	FAVIET	DFAVIET	515.124.4169	2002-08-16
110	JOHN	CHEN	JCHEN	515.124.4269	2005-09-28
111	ISMAEL	SCIARRA	ISCIARRA	515.124.4369	2005-09-30
112	JOSE MANUEL	URMAN	JMURMAN	515.124.4469	2006-03-07
113	LUIS	POPP	LPOPP	515.124.4567	2007-12-07
114	DEN	RAPHAELY	DRAPHEAL	515.127.4561	2002-12-07
115	ALEXANDER	KHOO	AKHOO	515.127.4562	2003-05-18
116	SHELLI	BAIDA	SBAIDA	515.127.4563	2005-12-24
117	SIGAL	TOBIAS	STOBIAS	515.127.4564	2005-07-24
118	GUY	HIMURO	GHIMURO	515.127.4565	2006-11-15
119	KAREN	COLMENARES	KCOLMENA	515.127.4566	2007-08-10
120	MATTHEW	WEISS	MWEISS	650.123.1234	2004-07-18
121	ADAM	FRIPP	AFRIPP	650.123.2234	2005-04-10

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NBR	HIRE_DATE
102	LEX	DE HAAN	LDEHAAN	515.123.4569	2001-01-13
109	DANIEL	FAVIET	DFAVIET	515.124.4169	2002-08-16
108	NANCY	GREENBERG	NGREENBE	515.124.4569	2002-08-17
114	DEN	RAPHAELY	DRAPHEAL	515.127.4561	2002-12-07
115	ALEXANDER	KHOO	AKHOO	515.127.4562	2003-05-18
100	STEVEN	KING	SKING	515.123.4567	2003-06-17
120	MATTHEW	WEISS	MWEISS	650.123.1234	2004-07-18
121	ADAM	FRIPP	AFRIPP	650.123.2234	2005-04-10
105	DAVID	AUSTIN	DAUSTIN	590.423.4569	2005-06-25
117	SIGAL	TOBIAS	STOBIAS	515.127.4564	2005-07-24
101	NEENA	KOCHHAR	NKOCHHAR	515.123.4568	2005-09-21
110	JOHN	CHEN	JCHEN	515.124.4269	2005-09-28
111	ISMAEL	SCIARRA	ISCIARRA	515.124.4369	2005-09-30
116	SHELLI	BAIDA	SBAIDA	515.127.4563	2005-12-24
106	VALLI	PATABALLA	VPATABAL	590.423.4560	2006-02-05
112	JOSE MANUEL	URMAN	JMURMAN	515.124.4469	2006-03-07
118	GUY	HIMURO	GHIMURO	515.127.4565	2006-11-15
107	DIANA	LORENTZ	DLORENTZ	590.423.5567	2007-02-07
119	KAREN	COLMENARES	KCOLMENA	515.127.4566	2007-08-10
113	LUIS	POPP	LPOPP	515.124.4567	2007-12-07

What are micro-partitions in Snowflake ?

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NBR	HIRE_DATE
102	LEX	DE HAAN	LDEHAAN	515.123.4569	2001-01-13
109	DANIEL	FAVIET	DFAVIET	515.124.4169	2002-08-16
108	NANCY	GREENBERG	NGREENBE	515.124.4569	2002-08-17
114	DEN	RAPHAELY	DRAPHEAL	515.127.4561	2002-12-07
115	ALEXANDER	KHOO	AKHOO	515.127.4562	2003-05-18
100	STEVEN	KING	SKING	515.123.4567	2003-06-17
120	MATTHEW	WEISS	MWEISS	650.123.1234	2004-07-18
121	ADAM	FRIPP	AFRIPP	650.123.2234	2005-04-10
105	DAVID	AUSTIN	DAUSTIN	590.423.4569	2005-06-25
117	SIGAL	TOBIAS	STOBIAS	515.127.4564	2005-07-24
101	NEENA	KOCHHAR	NKOCHHAR	515.123.4568	2005-09-21
110	JOHN	CHEN	JCHEN	515.124.4269	2005-09-28
111	ISMAEL	SCIARRA	ISCIARRA	515.124.4369	2005-09-30
116	SHELLI	BAIDA	SBAIDA	515.127.4563	2005-12-24
106	VALLI	PATABALLA	VPATABAL	590.423.4560	2006-02-05
112	JOSE MANUEL	URMAN	JMURMAN	515.124.4469	2006-03-07
118	GUY	HIMURO	GHIMURO	515.127.4565	2007-11-15
107	DIANA	LORENTZ	DLORENTZ	590.423.5567	2007-02-07
119	KAREN	COLMENARES	KCOLMENA	515.127.4566	2007-08-10
113	LUIS	POPP	LPOPP	515.124.4567	2007-12-07

METADATA	MIN	MAX
EMPLOYEE_ID	102	114
FIRST_NAME	LEX	DEN
LAST_NAME	DE HAN	RAPHAELY
EMAIL	LDEHAAN	DRAPHEAL
PHONE_NBR	515.123.4569	515.127.4561
HIRE_DATE	2001-01-13	2002-12-07
ROWS	4	

102	109
108	114
LEX	DANIEL
NANCY	DEN
DE HAAN	FAVIET
GREENBERG	RAPHAELY
LEDHAAN	DFAVIET
NGREENBE	DRAPHEAL
515.123.4569	515.124.4169
515.124.4569	515.127.4561
2001-01-13	2002-08-16
2002-08-17	2002-12-07

METADATA	MIN	MAX
EMPLOYEE_ID	100	120
FIRST_NAME	ADAM	STEVEN
LAST_NAME	FRIPP	WEISS
EMAIL	AFRIPP	SKING
PHONE_NBR	515.123.4567	650.123.2234
HIRE_DATE	2003-05-18	2005-04-10
ROWS	4	

115	100
120	121
ALEXANDER	STEVEN
MATTHEW	ADAM
KHOO	KING
WEISS	FRIPP
AKHOO	SKING
MWEISS	AFRIPP
515.127.4562	515.123.4567
650.123.1234	650.123.2234
2003-05-18	2003-06-17
2004-07-18	2005-04-10

What are micro-partitions in Snowflake ?

METADATA	MIN	MAX
EMPLOYEE_ID	102	114
FIRST_NAME	LEX	DEN
LAST_NAME	DE HAN	RAPHAELY
EMAIL	LDEHAAN	DRAPHEAL
PHONE_NBR	515.123.4569	515.127.4561
HIRE_DATE	2001-01-13	2002-12-07
ROWS	4	

METADATA	MIN	MAX
EMPLOYEE_ID	100	120
FIRST_NAME	ADAM	STEVEN
LAST_NAME	FRIPP	WEISS
EMAIL	AFRIPP	SKING
PHONE_NBR	515.123.4567	650.123.2234
HIRE_DATE	2003-05-18	2005-04-10
ROWS	4	

METADATA	MIN	MAX
EMPLOYEE_ID	101	117
FIRST_NAME	DAVID	SIGAL
LAST_NAME	AUSTIN	TOBIAS
EMAIL	DAUSTIN	STOBIAS
PHONE_NBR	515.123.4568	515.123.4568
HIRE_DATE	2005-06-25	2005-09-28
ROWS	4	

METADATA	MIN	MAX
EMPLOYEE_ID	106	112
FIRST_NAME	VALLI	JOSE MANUEL
LAST_NAME	PATABAL	URMAN
EMAIL	VPATABAL	JMURMAN
PHONE_NBR	590.423.4560	515.124.4469
HIRE_DATE	2005-09-30	2006-03-07
ROWS	4	

METADATA	MIN	MAX
EMPLOYEE_ID	107	119
FIRST_NAME	DIANA	LUIS
LAST_NAME	COLMENARES	POPP
EMAIL	DLORENTZ	LPOPP
PHONE_NBR	515.124.4567	515.124.4567
HIRE_DATE	2007-11-15	2007-12-07
ROWS	4	

HIRE_DATE
2001-01-13
2002-08-16
2002-08-17
2002-12-07
2003-05-18
2003-06-17
2004-07-18
2005-04-10
2005-06-25
2005-07-24
2005-09-21
2005-09-28
2005-09-30
2005-12-24
2006-02-05
2006-03-07
2007-11-15
2007-02-07
2007-08-10
2007-12-07

102	109
108	114
LEX	DANIEL
NANCY	DEN
DE HAAN	FAVIET
GREENBERG	RAPHAELY
LEDHAAN	DFAVIET
NGREENBE	DRAPHEAL
515.123.4569	515.124.4169
515.124.4569	515.127.4561
2001-01-13	2002-08-16
2002-08-17	2002-12-07

115	100
120	121
ALEXANDER	STEVEN
MATTHEW	ADAM
KHOO	KING
WEISS	FRIPP
AKHOO	SKING
MWEISS	AFRIPP
515.127.4562	515.123.4567
650.123.1234	650.123.2234
2003-05-18	2003-06-17
2004-07-18	2005-04-10

105	117
101	110
DAVID	SIGAL
NEENA	JOHN
AUSTIN	TOBIAS
KOCHHAR	CHEN
DAUSTIN	STOBIAS
NKOCHHAR	JCHEN
590.423.4569	515.127.4564
515.123.4568	515.124.4269
2005-06-25	2005-07-24
2005-09-21	2005-09-28

111	116
106	112
ISMAEL	SHELLI
VALLI	JOSE MANUEL
SCIARRA	BAIDA
PATABALLA	URMAN
ISCIARRA	SBAIDA
VPATABAL	JMURMAN
515.124.4369	515.127.4563
590.423.4560	515.124.4469
2005-09-30	2005-12-24
2006-02-05	2006-03-07

118	107
119	113
GUY	DIANA
KAREN	LUIS
HIMURO	LORENTZ
COLMENARES	POPP
GHIMURO	DLORENTZ
KCOLMENA	LPOPP
515.127.4565	590.423.5567
515.127.4566	515.124.4567
2007-11-15	2007-02-07
2007-08-10	2007-12-07

What are micro-partitions in Snowflake ?

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NBR	HIRE_DATE
102	LEX	DE HAAN	LDEHAAN	515.123.4569	2001-01-13
109	DANIEL	FAVIET	DFAVIET	515.124.4169	2002-08-16
108	NANCY	GREENBERG	NGREENBE	515.124.4569	2002-08-17
114	DEN	RAPHAELY	DRAPHEAL	515.127.4561	2002-12-07
115	ALEXANDER	KHOO	AKHOO	515.127.4562	2003-05-18
100	STEVEN	KING	SKING	515.123.4567	2003-06-17
120	MATTHEW	WEISS	MWEISS	650.123.1234	2004-07-18
121	ADAM	FRIPP	AFRIPP	650.123.2234	2005-04-10
105	DAVID	AUSTIN	DAUSTIN	590.423.4569	2005-06-25
117	SIGAL	TOBIAS	STOBIAS	515.127.4564	2005-07-24
101	NEENA	KOCHHAR	NKOCHHAR	515.123.4568	2005-09-21
110	JOHN	CHEN	JCHEN	515.124.4269	2005-09-28
111	ISMAEL	SCIARRA	ISCIARRA	515.124.4369	2005-09-30
116	SHELLI	BAIDA	SBAIDA	515.127.4563	2005-12-24
106	VALLI	PATABALLA	VPATABAL	590.423.4560	2006-02-05
112	JOSE MANUEL	URMAN	JMURMAN	515.124.4469	2006-03-07
118	GUY	HIMURO	GHIMURO	515.127.4565	2007-11-15
107	DIANA	LORENTZ	DLORENTZ	590.423.5567	2007-02-07
119	KAREN	COLMENARES	KCOLMENA	515.127.4566	2007-08-10
113	LUIS	POPP	LPOPP	515.124.4567	2007-12-07

Clustering Tables

Clustering tables (manual partitioning) is the process of explicitly telling Snowflake which column to use to create micro-partitions.

A table is provided a cluster key(Column Name) during creation based on which Snowflake will cluster(create micro-partitions) the table instead of automatically creating micro-partitions.

Why create a Cluster Key ?

- Snowflake generated micro-partitions are not providing sufficient query performance.
- Efficient query performance for large tables where queries frequently filter or aggregate data based on specific columns.

Features of Cluster Keys

- Only 1 Cluster key can be defined on a table, however a cluster key can have more than 1 column in it.
- If there is more than 1 column in the cluster key, the data in the micro-partition is organized by the first column and then by the second column within the first column.
- Cluster key can be created by using the CLUSTER BY clause in the create table statement.
- Cluster key can be added to a table using the ALTER TABLE STATEMENT post creation.

Table Clustering

Features & Characteristics of Cluster Keys:

- Tables greater than 1 TB can be good candidates for clustering.
- If query performance on a table with a snowflake defined micro-partition is not affected there is no need to cluster a table no matter what its size.
- Frequent updates and deletes disrupts clustering and micro-partition data. Snowflake works to re-clustering the data in the partition as per the cluster key in the background after updates and inserts. This process of re-clustering is called Automatic clustering and is enabled by default.
- Automatic clustering consumes serverless Snowflake credits as Snowflake works to keep the data in sync.
- You can disable automatic clustering by using ALTER TABLE and setting `AUTOMATIC_CLUSTERING` to false.
- You can re-enable automatic clustering by using ALTER TABLE and setting `AUTOMATIC_CLUSTERING` it to TRUE.
- Larger tables require more compute resources to cluster as there is more data resulting in higher credit consumption.
- A complex clustering key that involves multiple columns or a column with a lot of distinct values will require more processing leading to higher credit consumption.

Clustered Materialized Views

Why should Materialized views be clustered ?

Materialized views can be clustered just like tables by using the **CLUSTER BY** Clause.

The main purpose of clustering a Materialized views is to enhance query or select performance by creating alternate cluster keys which are different from those defined on the table.

Alternate cluster keys with Materialized views enable access to exactly the same data as the table via an alternate path.

```
CREATE OR REPLACE TABLE EMPLOYEES
(
  EMPLOYEE_ID      NUMBER(6),
  FIRST_NAME       VARCHAR(20),
  LAST_NAME        VARCHAR(25),
  EMAIL            VARCHAR(25),
  PHONE_NUMBER     VARCHAR(20),
  HIRE_DATE        DATE,
  JOB_ID           VARCHAR(10),
  SALARY           NUMBER(8,2),
  COMMISSION_PCT   NUMBER(2,2),
  MANAGER_ID       NUMBER(6),
  DEPARTMENT_ID    NUMBER(4)
)
CLUSTER BY (DEPARTMENT_ID, JOB_ID);
```

```
SELECT COUNT(*), SUM(SALARY) FROM EMPLOYEES
WHERE DEPARTMENT_ID =50
AND    JOB_ID='ST_CLERK'
```

```
SELECT COUNT(*), SUM(SALARY) FROM EMPLOYEES
WHERE SALARY >2000
```

Clustered Materialized Views

```
CREATE OR REPLACE MATERIALIZED VIEW MV_EMPLOYEE_SALARY
CLUSTER BY (SALARY)
AS
SELECT
    EMPLOYEE_ID ,
    FIRST_NAME ,
    LAST_NAME ,
    EMAIL ,
    PHONE_NUMBER ,
    HIRE_DATE ,
    JOB_ID ,
    SALARY ,
    COMMISSION_PCT ,
    MANAGER_ID ,
    DEPARTMENT_ID
FROM HRMS.HR.EMPLOYEES;
```

```
SELECT COUNT(*), SUM(SALARY) FROM MV_EMPLOYEES_SALARY
WHERE SALARY >2000;
```


Clustered Materialized Views

```
CREATE OR REPLACE MATERIALIZED VIEW MV_EMPLOYEE_EMAIL_PHONE
CLUSTER BY (EMAIL,PHONE_NUMBER)
AS
SELECT
    EMPLOYEE_ID ,
    FIRST_NAME ,
    LAST_NAME ,
    EMAIL ,
    PHONE_NUMBER ,
    HIRE_DATE ,
    JOB_ID ,
    SALARY ,
    COMMISSION_PCT ,
    MANAGER_ID ,
    DEPARTMENT_ID
FROM HRMS.HR.EMPLOYEES
```

```
SELECT COUNT(*) FROM MV_EMPLOYEE_EMAIL_PHONE
WHERE EMAIL LIKE '@GMAIL.COM' AND PHONE_NUMBER LIKE '847*'
```

Scaling up vs Scaling out.

Scaling up

Scaling up refers to increasing the size of a virtual warehouse (e.g., moving from XS to XL) to gain more compute resources, such as CPU, memory, and I/O. This allows for faster query processing, better performance for complex queries and the ability to handle larger volumes of data.

In scaling up the number of virtual warehouse that are in action does not change, instead the power or capacity of an existing warehouse is increased.

Scaling up is typically used when a specific workload requires more computational power per node, such as complex queries, large aggregations, or data transformations that benefit from higher CPU or memory availability.

Scaling up can result in faster query performance because each node has more resources.

Scaling up vs Scaling out.

Scenario

Flipkart, a major e-commerce company, is hosting a massive New Year online sale. The company anticipates a 10X increase in data volume due to a surge in customer transactions, product searches, and real-time inventory updates. This data needs to be ingested into Snowflake for immediate analysis and reporting, with complex computations required to generate insights on sales trends, customer behavior, and inventory management.

Challenge

- The drastic increase in data volume and the complexity of the required computations put significant demand on the database's memory and compute resources.
- Flipkart needs to ensure that the data is processed quickly and efficiently, with minimal latency in report generation, to support real-time decision-making during the sale.

Solution: Scaling Up

Increased Compute and Memory Resources.

By scaling up to a larger Snowflake warehouse (e.g., moving from L to 2XL), Flipkart can handle the massive increase in data volume and the associated complex computations. The additional compute resources allow for faster data processing and helps prevent delay due to increase in data volume.

Scaling up vs Scaling out.

Improved Performance for Complex Queries.

With the larger warehouse, complex queries that involve heavy computations, such as aggregations, joins, and filtering across large datasets, can be executed more efficiently. This ensures that the analytics team can generate real-time insights without delays.

Handling Peak Loads.

The larger warehouse can absorb the increased load during the peak of the sale, preventing potential slowdowns or bottlenecks that could occur with a smaller warehouse.

Summary.

By scaling up, Flipkart effectively managed the increased data volume and complex computations during the New Year sale, ensuring that the analytics and reporting processes remain fast and reliable. This enabled the company to make timely, data-driven decisions that enhance the customer experience and optimize sales performance during the critical sale period.

This example clearly illustrates how scaling up can be a response to a significant, temporary increase in data volume and computational demands.

Scaling up vs Scaling out.

Scaling out

Scaling out in Snowflake refers to increasing the number of virtual warehouses rather than increasing the size of a single warehouse.

Scaling out adds more virtual warehouses, which allows it to handle more concurrent queries without overloading a single warehouse.

Scaling out is beneficial when you have a large number of simple queries that need to be processed in parallel. It's particularly useful for workloads where the ability to run multiple simple queries simultaneously is critical.

Scaling out can significantly increase throughput and reduce query times by distributing the workload across more virtual warehouses.

Scaling out in Snowflake is handled by enabling multi-cluster in warehouses which adds virtual warehouses into a pool of similar sized warehouses all of which can be used to handle query processing.

Scaling up vs Scaling out.

Scenario

Walmart is suddenly experiencing a busy evening and the number of customers in the store is far higher than usual. As more shoppers fill their carts and head to the checkout area, the lines start to grow.

Challenge

With only a few checkout counters open, the wait times increase as more customers try to complete their purchases. This is leading to long lines and a poor shopping experience.

Solution

The scale out solution to this problem is Opening Additional Checkout Counters To handle the surge in customers, As more counters open, the flow of customers is distributed across all available counters. This reduces the wait time for each customer.

Scaling up vs Scaling out.

The MAX_CONCURRENCY_LEVEL parameter which is set to 8 by default for a warehouse is the maximum number of queries that can be handled simultaneously by a single warehouse.

1 warehouses = 8 Queries
2 warehouses = 16 Queries
3 warehouses = 24 Queries
4 warehouses = 32 Queries

As the number of WH increases the capacity to process queries simultaneously goes up.

In Snowflake scale up and scale out strategies can often be combined depending on the needs of the workload.

What is a scaling policy ?

A scaling policy in Snowflake refers to how quickly a virtual warehouses will be provisioned when there is a demand for compute because of queries that need to be processed.

The scaling policy also determines how quickly a virtual warehouse will be de-provisioned if the number of queries waiting to be processed fall below a certain threshold.

Scaling policy comes into play when multi-cluster warehouse is enabled and operating in Auto-Scale mode.

Scaling policy affects

- How quickly a query is processed.
- Amount of credits consumed.

Types of Scaling policy

- Standard (Default).
- Economy.

What is a scaling policy ?

Policy	Description	Warehouse Starts...	Warehouse Shuts Down...
Standard (default)	Prevents/minimizes queuing by favoring starting additional clusters over conserving credits.	The first cluster starts immediately when either a query is queued or the system detects that there's one more query than the currently-running clusters can execute. Each successive cluster waits to start 20 seconds after the prior one has started. For example, if your warehouse is configured with 10 max clusters, it can take a full 200+ seconds to start all 10 clusters.	After 2 to 3 consecutive successful checks (performed at 1 minute intervals), which determine whether the load on the least-loaded cluster could be redistributed to the other clusters without spinning up the cluster again.
Economy	Conserves credits by favoring keeping running clusters fully-loaded rather than starting additional clusters, which may result in queries being queued and taking longer to complete.	Only if the system estimates there's enough query load to keep the cluster busy for at least 6 minutes.	After 5 to 6 consecutive successful checks (performed at 1 minute intervals), which determine whether the load on the least-loaded cluster could be redistributed to the other clusters without spinning up the cluster again.

What is a scaling policy ?

Stable workloads with short spike or increase in demand for compute for a short duration should use Economy scaling policy.

Variable workloads or workloads that must be processed immediately should use Standard scaling policy.

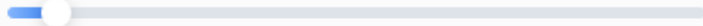
What is Query Acceleration Service ?

Query Acceleration service is a feature of Snowflake which can be enabled at the warehouse level to improve complex query performance by providing serverless compute to process queries which need additional compute to accelerate query processing.

```
CREATE OR REPLACE WAREHOUSE SCALE_LAB_WH  
WITH  
WAREHOUSE_SIZE = 'XSMALL'  
AUTO_SUSPEND = 5  
AUTO_RESUME = TRUE  
INITIALLY_SUSPENDED = TRUE  
ENABLE_QUERY_ACCELERATION = TRUE
```

☐ Multi-cluster Warehouse
Scale compute resources as query concurrency needs change. [Learn more](#)

☒ Query Acceleration
Accelerate outlier queries with additional flexible compute resources. [Learn more](#)

Scale Factor ⓘ x8


```
ALTER WAREHOUSE SCALE_LAB_WH SET ENABLE_QUERY_ACCELERATION = TRUE;
```

What is Scale factor in Query Acceleration Service?

Scale factor is a cost control mechanism that allows you to set an upper limit on the amount of compute resources a warehouse can use for Query Acceleration Service(QAS).

Scale Factor	Size	WH Credits	QAS Credits
1	XSMALL	1	1
2	XSMALL	1	2
1	SMALL	2	2
2	SMALL	2	4
1	MEDIUM	4	4
2	MEDIUM	4	8

$$\text{QAS Credits/Hour} = \text{Scale Factor} * \text{WH Credits/Hour}$$

Unlimited Query Acceleration credits

QUERY_ACCELERATION_MAX_SCALE_FACTOR = 0

```
CREATE OR REPLACE WAREHOUSE SCALE_LAB_WH
WITH
WAREHOUSE_SIZE = 'XSMALL'
AUTO_SUSPEND = 5
AUTO_RESUME = TRUE
INITIALLY_SUSPENDED = TRUE
ENABLE_QUERY_ACCELERATION = TRUE
QUERY_ACCELERATION_MAX_SCALE_FACTOR = 2
```

Summary

- Data caching is the process of temporarily storing query results or data for reuse by other queries leading to time and credit saving and efficient use of resources.
- Types of cache available in Snowflake are
 - Result cache
 - Local disk Cache
 - Metadata cache
- Results cache stores the results of queries that have been previously executed.
- An identical query submitted again will use the cache if the underlying data hasn't changed and the order of the columns or case in the query has not changed and queries do not have functions that produce nondeterministic results like CURRENT_TIMESTAMP
- Fetching data from the results cache does not consume credits and is much faster.
- The Local disk cache stores data that has been read from cloud storage during query execution.(Eg: COPY)
- Local disk cache is in the SSD(Hard Drive) storage of the virtual warehouse.
- The local disk cache persists as long as the virtual warehouse is running. The local disk cache is lost when the warehouse is suspended or deleted.
- The metadata cache stores information like table names and their column names in the cache.
- Table statistics like number of rows in the table, MIN and MAX values for each column are stored and Object dependencies are stored in the metadata cache
- Metadata cache reduces time needed to parse the query as references to tables and columns are available in the cache and do not need to go to the database layer to fetch data.

Summary

- Clustering tables (manual partitioning) is the process of explicitly telling Snowflake which column to use to create micro-partitions.
- Cluster keys are created if Snowflake generated micro-partitions are not providing sufficient query performance.
- Materialized views can be clustered just like tables by using the CLUSTER BY Clause.
- The main purpose of clustering a Materialized views is to enhance query or select performance by creating alternate cluster keys which can be used for querying the same data
- Any number of Materialized views can be created on a table.
- Tables greater than 1 TB can be good candidates for Clustering.
- Snowflake works to re-clustering the data in the partition as per the cluster key in the background and this consumes serverless credits.
- **Scaling up refers to increasing the size of a virtual warehouse (e.g., moving from XS to XL) to gain more compute resources such as CPU memory and I/O to allow for faster query processing , better performance for complex queries and to handle larger volumes of data.**
- In scaling up the number of virtual warehouse that are in action does not change, instead the power or capacity of an existing warehouse is increased.
- Scaling up is typically used when a specific workload requires more computational power.
- Scaling out in Snowflake refers to increasing the number of active virtual warehouses rather than increasing the size of a single warehouse.
- Scaling out adds more virtual warehouses which allows it to handle more concurrent queries without overloading a single warehouse.

Summary

- The MAX_CONCURRENCY_LEVEL parameter which is set to 8 by default for a warehouse is the maximum number of queries that can be handled simultaneously by a single warehouse.
- A scaling policy in Snowflake refers to how quickly a virtual warehouses will be provisioned when there is a demand for compute because of queries that need to be processed.
- Two types of scaling policy are supported Standard (Default) and Economy.
- Query Acceleration service is a feature of Snowflake which can be enabled at the warehouse level to improve complex query performance by providing serverless compute to process queries which need additional compute to accelerate query processing.
- Scale factor is a cost control mechanism that allows you to set an upper limit on the amount of compute resources a warehouse can use for Query Acceleration Service(QAS).
- Query Acceleration service can be enabled either using SQL or Snowsight.

Thank you



Learn2
CloudData

Powered by

www.learn2clouddata.com

Find Us

www.learn2clouddata.com